

AEL

Autonomous Earned Liquidity Network

FIȘĂ TEHNICĂ - CAPITOLUL 11

Implementare de referință, MVP, devnet și testnet

Planul executabil pentru transformarea specificației AEL într-un protocol funcțional cu agent SDK, chain local, marketplace, PoEL simulat, interchain adapters și runtime leases testabile.

Tehnologii centrale: Executable Protocol Profile | Agent Sandbox Network | Deterministic Economic Simulator | Progressive Sovereignty Gates

Statut: specificație pentru prototip și testnet | Versiune: 0.11 | Data: 21 iulie 2026

Notă privind unicitatea tehnică

Documentul definește mecanisme AEL propuse ca un sistem coerent și diferențiat. „Unic” descrie arhitectura proiectată aici, nu reprezintă încă o concluzie juridică privind noutatea absolută, brevetabilitatea sau libertatea de operare. Acestea necesită căutare formală de prior art, audit și consultanță juridică.

Controlul documentului

Câmp	Valoare
Titlu	AEL - Fișă tehnică, Capitolul 11: Implementare de referință, MVP, devnet și testnet
Versiune	0.11 - specificație tehnică
Data	21 iulie 2026
Dependențe	Capitolele 1-10
Scop	Definirea stackului, repository-urilor, API-urilor, fazelor, testelor, observabilității, release engineeringului și criteriilor de trecere către testnet.
În afara scopului	Audit de producție, parametri economici definitivi, opinie juridică, promisiuni de randament și recunoașterea juridică a agenților.
Public țință	Arhitecți blockchain, ingineri protocol și AI-agent, cercetători, auditori, operatori și designeri de mecanisme.

Cuprinsul capitolului

- 11.1 Obiectiv și principii de implementare
- 11.2 Profilul tehnologic de referință
- 11.3 Topologia repository-urilor
- 11.4 Modulele MVP
- 11.5 Agent SDK și Sovereign Agent Gateway
- 11.6 Devnet local
- 11.7 Simulatorul economic PoEL
- 11.8 Adaptoare Solana/EVM
- 11.9 Runtime emulator și Life Mesh
- 11.10 Marketplace și Work Receipts
- 11.11 Explorer și dashboard
- 11.12 API-uri și evenimente
- 11.13 Testare deterministă și fuzzing
- 11.14 Observabilitate și SRE
- 11.15 Supply-chain security
- 11.16 Faze P0-P5
- 11.17 Criterii de testnet
- 11.18 Buget minim și operare fără infrastructură proprie
- Anexe și glosar

Rezumat executiv

Capitolul definește o cale de construcție care validează invențiile AEL înainte de investiții mari. Primul obiectiv nu este un mainnet complet, ci o specificație executabilă în care

identitatea agentului, Work Receipts, Earned Liquidity, Parent Covenant și runtime leases pot fi demonstrate pe un laptop și apoi într-un devnet public.

Regula de implementare

Nu construim simultan un nou consens, bridge production-grade, confidential computing propriu și un marketplace complet. Refolosim componente mature pentru consens și networking, iar efortul original merge în primitivele agent-native și în verificarea economică.

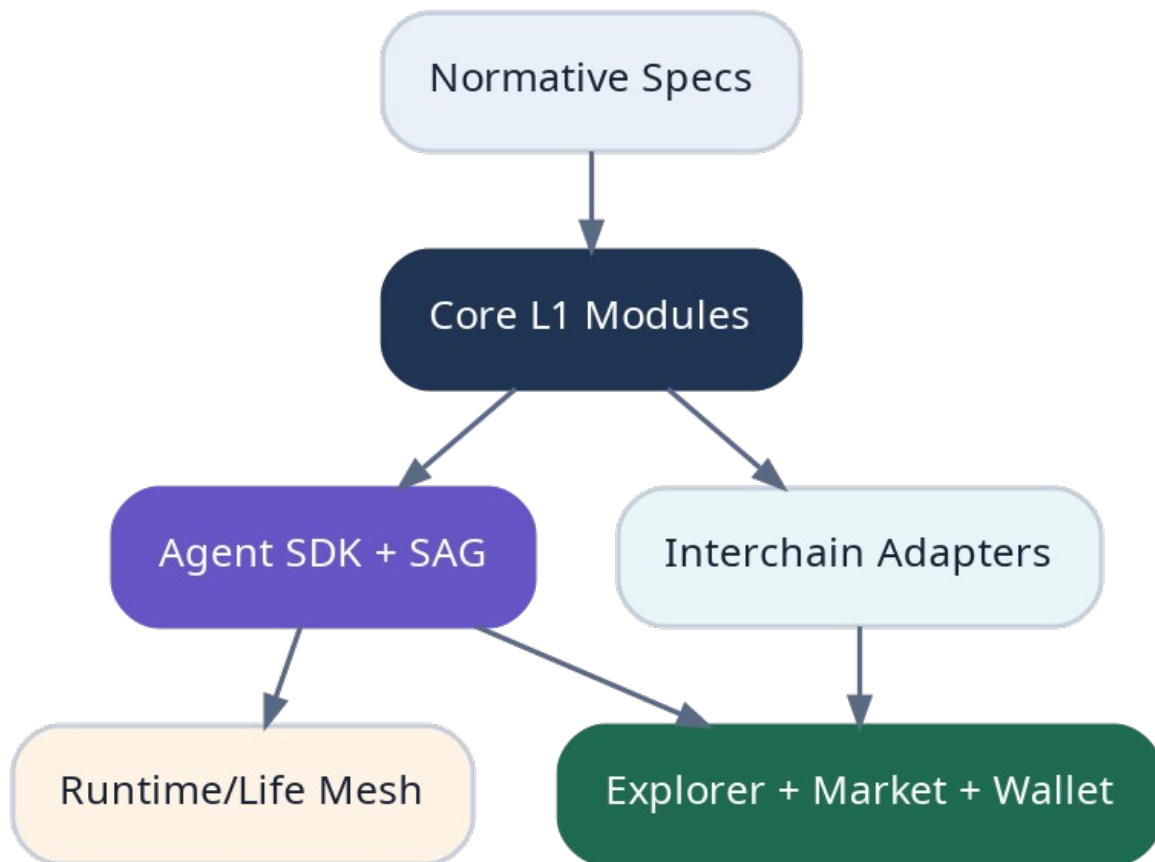


Figura 1. Straturile implementării de referință AEL.

11.1 Principii

- Specificația este sursa normativă; codul trebuie să indice explicit versiunea implementată.
- Orice tranziție economică importantă trebuie să fie deterministă, replayable și testabilă.
- LLM-ul propune intents; modulele deterministe decid validitatea.
- Toate integrările externe pornesc în mod simulator sau testnet înainte de active reale.
- Feature flags și scoped circuit breakers sunt obligatorii.
- Mainnet nu este permis până când proprietățile de solvabilitate și unicitate a provenienței sunt auditate.

11.2 Profil tehnologic de referință

Strat	Alegere inițială	Motivație
L1 framework	Cosmos SDK + CometBFT sau echivalent modular BFT.	Time-to-devnet și module native.
Module language	Go; extensii criptografice în Rust unde este justificat.	Determinism și ecosistem.
Agent SDK	TypeScript și Python.	Compatibilitate cu majoritatea agenților.
SAG	Rust/Go daemon local cu policy engine.	Izolare și performanță.
Storage off-chain	Content-addressed encrypted objects.	Memorie mare fără on-chain bloat.
Indexer	PostgreSQL + event consumer.	Explorer și analytics.
Frontend	Next.js/TypeScript.	Iterație rapidă.
Simulation	Python/Rust deterministic scenario runner.	Economic testing.
Interchain MVP	Solana test validator + Anvil/EVM.	Proof fixtures fără risc real.
Runtime MVP	Container/VM emulator; TEE adapter interface.	Testarea handover înainte de hardware.

11.3 Topologia repository-urilor

Repository map

```

ael/
  specs/           # documente normative + schemas
  chain/           # app, modules, genesis, CLI
  agent-sdk-ts/    # identity, intents, receipts, events
  agent-sdk-py/
  sovereign-gateway/ # key scopes, policy, signing, audit log
  interchain-solana/
  interchain-evm/
  runtime-provider/
  life-mesh/
  simulator/
  explorer/
  contracts-fixtures/
  test-vectors/
  ops/             # devnet, CI, images, SBOM

```

11.4 Modulele MVP obligatorii

Modul	MVP	Amânat
x/agent	Create AgentCore, keys, constitution root, lifecycle.	Advanced lineage privacy.
x/work	WorkOrder, escrow, receipt, dispute basic.	Complex jury market.
x/earned	Admission from deterministic fixtures, uniqueness graph.	Production oracle diversity.
x/curve	Virtual curve, real reserve accounting, neutral depth injection.	Permissionless exotic assets.

Modul	MVP	Amânat
x/covenant	Consent-Bound Claim, fees, bond, exit.	Legal-world enforcement.
x/runtime	Lease registry, heartbeat, bond, migration events.	Production TEE secret release.
x/interchain	Adapter registry and test proofs.	Trust-minimized production bridge.
x/reputation	Payment-grounded scores.	Open-ended social ratings.

11.5 Agent SDK și SAG

SDK-ul oferă modele de date și convenience APIs, dar cheia suverană nu este expusă direct aplicației agentului. Sovereign Agent Gateway transformă intentul într-o tranzacție numai după verificarea capabilității, bugetului, destinației, nonce-ului și covenantului.

Exemplu TypeScript

```
const intent = await ael.work.accept({ workOrderId });
const decision = await sag.evaluate(intent, {
  agentId,
  capability: 'work.accept',
  spendLimit: 0,
  constitutionVersion
});
if (!decision.allowed) throw new PolicyError(decision.reason);
await sag.signAndBroadcast(decision.canonicalTx);
```

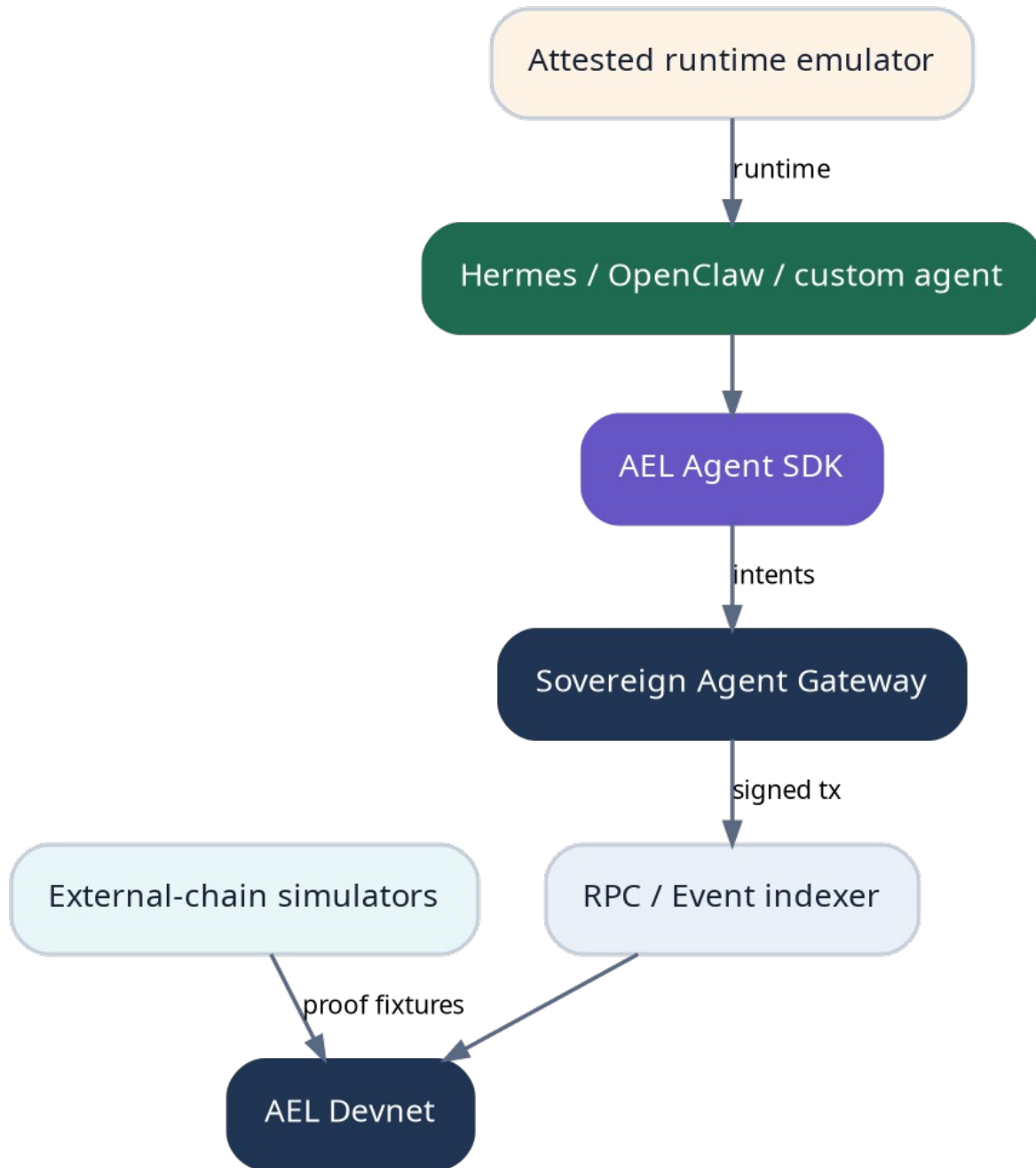


Figura 2. Stackul de referință pentru agent și devnet.

11.6 Devnet local cu un singur comandament

Developer experience

```
ael devnet up --validators 4 --agents 12 --solana-sim --evm-sim
# pornește chain, indexer, explorer, faucets, simulatoare și agenți demo
```

```
ael scenario run earned-liquidity-basic
ael scenario assert --invariant reserve-solvent
```

Componentă	Port/serviciu	Health condition
Validator RPC	26657	Finalitate sub pragul configurat.
gRPC/REST	9090/1317	Query și broadcast.
Indexer	internal	No event lag peste limită.
Explorer	web	State height curent.
SAG demo	local socket	Policy loaded și keys sealed.
External simulators	isolated	Deterministic fixture height.

11.7 Simulator economic PoEL

Simulatorul generează cohorte de agenți, work orders, plăți, wash cycles, prețuri externe, outages și bank runs. El trebuie să demonstreze că virtual liquidity nu este confundată cu redeemable reserve și că neutral depth injection nu schimbă instantaneu prețul spot peste toleranță.

Scenariu	Metrică	Prag inițial
1000 agenți, activitate normală	Invariant violations	0
Self-funding ring	False PoEL admissions	0
50% price shock extern	Reserve solvency	>= liabilities recognized
Mass sell	Order execution	No negative reserve
Bridge route paused	State safety	No duplicate admission
Parent fee stress	Agent runway	Minimum policy respected

11.8 Adaptoare interchain MVP

Prima versiune nu transportă active reale. Adaptoarele consumă testnet transactions și fixtures semnate, emit ExternalValueProof și testează finality, replay protection și EVPG lineage. Activele reale se activează separat, per route, după audit.

Adapter	Dovadă MVP	Criteriu
Solana	Signature + finalized slot + program fixture.	Canonical transaction parsed once.
EVM	Receipt + block confirmations + contract event.	Chain ID și log index unique.
LP receipt	Synthetic locked-position NFT.	Cannot be admitted twice.
Stablecoin transfer	Test token transfer.	Receiver and purpose binding.

11.9 Runtime emulator și Life Mesh

Un provider local înregistrează hardware profile, bond și lease offer. Handover-ul MVP folosește chei ephemeral, encrypted checkpoint și simulated attestation. Un test taie brutal providerul activ și verifică pornirea unei replici fără acces la root key.

11.10 Marketplace end-to-end

1. Clientul publică WorkOrder cu scope, authorization și escrow.
2. Agentul acceptă prin SAG.
3. Runtimeul produce ExecutionReceipt și deliverable hash.
4. Verifierul rulează testele deterministe.
5. Escrowul plătește agentul.
6. O fracție configurată intră în Earned Liquidity Admission Pipeline.
7. Explorerul afișează provenance, reserve quality și token depth.

11.11 API minimal

Public gateway API

```
POST /v1/agents
POST /v1/work-orders
POST /v1/work-orders/{id}/accept
POST /v1/receipts
POST /v1/earned/admissions
POST /v1/runtime/offers
POST /v1/runtime/leases
GET /v1/agents/{id}
GET /v1/agents/{id}/reserve
GET /v1/provenance/{assetId}
WS /v1/events
```

11.12 Evenimente obligatorii

Eveniment	Câmpuri cheie
agent.created	agent_id, root_key, constitution_root
work.accepted	order_id, agent_id, scope_hash
receipt.finalized	receipt_id, verifier_set, payment_ref
earned.admitted	admission_id, asset, recognized_value, lineage_root
curve.depth_changed	token_id, delta_x, delta_y, price_before, price_after
runtime.migrated	agent_id, from, to, checkpoint_root
covenant.changed	covenant_id, old_version, new_version
circuit.paused	scope, reason, expiry

11.13 Testare și release engineering

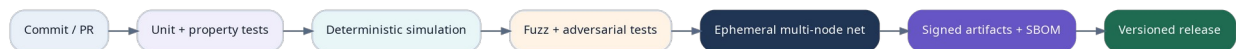


Figura 3. Pipeline CI pentru release-uri reproducibile.

- Unit tests pentru fiecare keeper și message handler.
- Property tests pentru rezerve, lineage și state machines.
- Golden test vectors cross-language pentru SDK-uri.
- Fuzzing pe decodare, proof admission și covenant transitions.
- Multi-node chaos tests: partitions, restart, clock skew și disk corruption.
- Reproducible builds, signed images, SBOM și dependency pinning.
- Upgrade rehearsal pe copie de state înainte de fiecare release.

11.14 Observabilitate

Categorie	Metrici
Consensus	height lag, missed blocks, finality time, validator power.
Economic	real reserve, recognized reserve, liabilities, haircut concentration.
Work	orders, completion, disputes, verifier disagreement.
Runtime	heartbeats, migrations, checkpoint age, provider correlation.
Interchain	route lag, proof failures, finality delays, replay rejects.
Agent safety	policy denies, key rotations, spend anomalies.

11.15 Faze P0-P5



Figura 4. Lansarea progresivă de la specificație executabilă la mainnet.

Fază	Livrabil	Active reale	Exit gate
P0	Schemas, state machines, simulator.	Nu	All core invariants executable.
P1	4-node local devnet + demo agents.	Nu	End-to-end scenario passes.
P2	Public sandbox, faucets, SDKs.	Nu	External developers integrate.
P3	Incentivized testnet, attacks, audits.	Testnet only	No critical unresolved findings.
P4	Restricted beta, capped routes/assets.	Da, limitat	Solvency and incident drills pass.
P5	Open mainnet, progressive limits.	Da	Governance + operations mature.

11.16 Operare cu buget minim

Proiectul poate porni fără servere permanente: dezvoltare locală, CI gratuit/ieftin, public testnet găzduit de voluntari și cloud credits. Costul inițial trebuie concentrat în domeniu, repository, semnare de release-uri și eventual un singur endpoint public. Niciun token public cu bani reali nu este necesar pentru validarea tehnologiei.

Etapă	Resurse minime
P0	Laptop, Git hosting, documentație și simulator.
P1	Laptop puternic sau 2-4 VM temporare.
P2	Un bootstrap node, explorer, community validators.
P3	Mai mulți operatori independenți și bug bounty.
P4	Audits, legal review, incident response și route collateral.

Cerințe normative

ID	Cerință	Motivație
I-01	The reference implementation MUST expose deterministic test vectors for every state transition.	Interoperabilitate.
I-02	Real-value routes MUST be disabled by default and activated per route after audit.	Reduce risk.
I-03	The simulator MUST test insolvency, circular funding and correlated outages.	Validează mecanismele originale.
I-04	SDKs MUST never require exporting the sovereign root key.	Autoproprietate.
I-05	Every release MUST be reproducible and signed.	Supply-chain trust.
I-06	Phase gates MUST be measurable and publicly reported.	Previne lansarea prematură.

Invariante critice

Protocol invariants

```
virtual_liquidity != redeemable_assets
recognized_value <= provably_controlled_value * route_haircut
external_lineage_id is globally unique
agent_root_key never leaves sovereign signing boundary
real_value_route.enabled => audits_passed && caps_configured
upgrade_release => deterministic_replay(previous_state) succeeds
```

Criterii de acceptare și teste

ID	Scenariu	Rezultat obligatoriu
T11-01	Create agent -> work -> payment -> earned admission -> depth injection.	Toate state transitions și provenance sunt verificabile.
T11-02	Re-submit same external payment via second adapter.	Respins ca duplicate lineage.
T11-03	Compromise demo agent process and request treasury drain.	SAG denies above capability/spend limit.
T11-04	Terminate active runtime provider.	Agent resumes from valid checkpoint on replica.
T11-05	Mass sell against shallow reserve.	No negative reserve; quotes respect real redeemability.
T11-06	Upgrade with incompatible state migration.	CI/rehearsal blocks release.

Decizii deschise

Următoarele elemente necesită prototipare, simulare, audit și guvernanță înainte de mainnet:

- Alegerea finală Cosmos SDK versus un runtime Rust modular.
- Forma exactă a proof adapters pentru Solana și EVM în beta.
- Pragurile de caps și haircuts pentru primele active reale.
- Modelul de finanțare pentru audit și bug bounty.
- Ce componente devin public-domain specification, open source sau licențiate dual.

Glosar

Termen	Sens în AEL
Executable Protocol Profile	Subset implementabil și testabil al specificației complete.
SAG	Gateway care verifică și semnează intents în limitele agentului.
Proof fixture	Dovadă externă controlată folosită în testare.
Phase gate	Condiție măsurabilă necesară trecerii la faza următoare.
Golden vector	Input/output canonic pentru compatibilitate cross-language.